

Ações CRUD no ASP.NET Core

Concluído

- 5 minutos

Nosso serviço de pizza dá suporte a operações CRUD para uma lista de pizzas.

Essas operações são executadas por meio de verbos HTTP, que são mapeados por meio de atributos ASP.NET Core. Como você viu, o verbo HTTP `GET` é usado para recuperar um ou mais itens de um serviço. Essa ação é anotada com o atributo `[HttpGet]`.

A seguinte tabela mostra o mapeamento das quatro operações que você está implementando para o serviço de pizza:

Verbo de ação HTTP	Operação CRUD	Atributo ASP.NET Core
<code>GET</code>	Leitura	<code>[HttpGet]</code>
<code>POST</code>	Criar	<code>[HttpPost]</code>
<code>PUT</code>	Atualizar	<code>[HttpPut]</code>
<code>DELETE</code>	Excluir	<code>[HttpDelete]</code>

Você já viu como as ações `GET` funcionam. Vamos saber mais sobre as ações `POST`, `PUT` e `DELETE`.

POST

Para permitir que os usuários adicionem um novo item ao ponto de extremidade, você deve implementar a ação `POST` usando o atributo `[HttpPost]`. Ao passar o item (neste exemplo, uma pizza) para o método como um parâmetro, o ASP.NET

Core converte automaticamente qualquer aplicativo/JSON enviado ao ponto de extremidade em um objeto de `Pizza` do .NET preenchido.

Está é a assinatura do método `Create` que você implementará na próxima seção:

C#

```
[HttpPost]
public IActionResult Create(Pizza pizza)
{
    // This code will save the pizza and return a result
}
```

O atributo `[HttpPost]` mapeia solicitações `POST` HTTP enviadas para `http://localhost:5000/pizza` usando o método `Create()`. Em vez de retornar uma lista de pizzas, como vimos com o método `Get()`, esse método retorna uma resposta `IActionResult`.

`IActionResult` informa ao cliente se a solicitação foi bem-sucedida e fornece a ID da pizza recém-criada. `IActionResult` usa códigos de status HTTP padrão, para que possa se integrar facilmente aos clientes, independentemente da linguagem ou da plataforma em que estão sendo executados.

ASP.NET Core resultado da ação	Código de status HTTP	Descrição
<code>CreatedAtAction</code>	201	A pizza foi adicionada ao cache na memória. A pizza está incluída no corpo

		da resposta no tipo de mídia, conforme definido no cabeçalho da solicitação HTTP <code>accept</code> (JSON por padrão).
<code>BadRequest</code> está implícito	400	O objeto <code>Pizza</code> do corpo da solicitação é inválido.

Felizmente, `ControllerBase` tem métodos de utilitário que criam as mensagens e os códigos de resposta HTTP apropriados para nós. Você verá como esses métodos funcionam no próximo exercício.

PUT

Modificar ou atualizar uma pizza em nosso inventário é semelhante ao método POST que você implementou, mas usa o atributo `[HttpPut]` e recebe o parâmetro `id` além do objeto `Pizza` que precisa ser atualizado:

C#

```
[HttpPut("{id}")]
public IActionResult Update(int id, Pizza pizza)
```

```
{  
    // This code will update the pizza and return a result  
}
```

Cada instância de `ActionResult` usada na ação anterior é mapeada para o código de status HTTP correspondente na tabela a seguir:

ASP.NET Core resultado da ação	Código de status HTTP	Descrição
<code>NoContent</code>	204	A pizza foi atualizada no cache na memória.
<code>BadRequest</code>	400	O valor <code>Id</code> do corpo da solicitação não corresponde ao valor <code>id</code> da rota.
<code>BadRequest</code> está implícito	400	O objeto <code>Pizza</code> do corpo da solicitação é inválido.

DELETE

Uma das ações mais fáceis de implementar é a ação `DELETE`, que leva apenas o parâmetro da pizza `id` para remover do cache na memória:

C#

```
[HttpDelete("{id}")]  
public IActionResult Delete(int id)  
{  
    // This code will delete the pizza and return a result  
}
```

Cada instância de `ActionResult` usada na ação anterior é mapeada para o código de status HTTP correspondente na tabela a seguir:

ASP.NET Core resultado da ação	Código de status HTTP	Descrição
<code>NoContent</code>	204	A pizza foi excluída do cache na memória.
<code>NotFound</code>	404	Uma pizza que corresponde ao parâmetro <code>id</code> fornecido não existe no

		cache na memória.
--	--	----------------------

O exercício da próxima unidade demonstra como dar suporte a cada uma das quatro ações na API da web.